

IT Automation Incident Ticket Manager

Objective

The objective of this program is to create a simple IT Automation Incident Ticket Manager that can organize and manage incident tickets created by automation bots. The program allows the user to add, display, search, remove, and count incident tickets.

Through this activity, I can apply the concepts I have learned about Object-Oriented Programming, especially classes, objects, encapsulation, and methods, while also using a linear data structure to store the incident tickets.

Scenario

In an IT environment, automation bots can create incident tickets when something goes wrong. These tickets contain information such as the Incident ID, the Bot that created the ticket, and a short description of the problem.

Instead of viewing the tickets separately, the program organizes them so they can be managed through different operations. The user can add a new incident ticket, view all active tickets, search for a specific ticket, remove a resolved ticket, and check how many active tickets remain.

The program was tested using the ten sample incident tickets provided in the laboratory activity.

Data Structure Used

The program uses a Python list as its linear data structure.

```
tickets = []
```

The list stores the incident ticket objects in a sequence. Each object contains the Incident ID, Bot, and Short Description.

The list allows the program to add new tickets, display the tickets, search for a ticket, remove a resolved ticket, and count the active tickets.

Program Operations

The program has a menu that allows the user to select the operation they want to perform.

1. Add Incident Ticket

The user can add a new incident ticket by entering the Incident ID, Bot, and Short Description. The program also includes a Random Ticket option that can generate a ticket automatically.

2. Display All Active Incident Tickets

This operation displays all the incident tickets currently stored in the program.

3. Search for a Specific Incident Ticket

The user enters an Incident ID, and the program searches through the tickets. If the Incident ID exists, the program displays the information of that ticket.

4. Remove a Resolved Incident Ticket

The user enters the Incident ID of a resolved ticket. The program searches for it and removes it from the active tickets.

5. Count Active Incident Tickets

This operation displays the total number of active incident tickets currently stored in the program.

6. Exit

This ends the program.

Source Code

The program was created using Python. It uses a class called IncidentTicket to represent each incident ticket.

The program also uses encapsulation by keeping the ticket information inside the class and providing a method to access the Incident ID.

The complete source code is included in the .py file submitted inside the LAB EXER 1 folder.

Program Output

The program was tested using the ten sample incident tickets provided in the laboratory activity.

The following operations were demonstrated:

- Adding an incident ticket
- Displaying all active incident tickets
- Searching for an incident ticket
- Removing a resolved incident ticket
- Counting active incident tickets
- Using the program menu

Conclusion

This laboratory exercise helped me understand how a simple program can be used to manage incident tickets in an IT-related scenario. By creating the Incident Ticket Manager, I was able to apply classes, objects, encapsulation, and methods to something that can represent an actual IT support situation.

The program also showed me how a linear data structure can be used to keep multiple records organized. Instead of only creating something that works, I was able to understand how the different parts of the program connect with each other, from creating an incident ticket object to adding, searching, displaying, removing, and counting the tickets.

Through this activity, I can see how programming concepts can be used to create something beyond simply writing code, especially when the goal is to organize information and make a process easier to manage.